

Social Seller — Sprint 1

Guide de Setup · Expo + Supabase + Claude Code · Semaines 1–2

Ce guide couvre les **10 jours du Sprint 1** : setup de l'environnement, architecture du projet, schéma base de données multi-tenant, et implémentation des US-01 à US-05. Chaque étape inclut les **prompts Claude Code** prêts à copier-coller.

Couche	Technologie	Rôle	Lien
Mobile	Expo (React Native)	App iOS + Android	expo.dev
Langage	TypeScript	Typage statique, moins de bugs	—
Navigation	Expo Router	Navigation entre écrans	—
Backend	Supabase	BDD + Auth + Realtime + Storage	supabase.com
Webhooks	Node.js + Express	Réception APIs réseaux sociaux	railway.app
IA Dev	Claude Code	Génération de code assistée	claude.ai/code

Jour 1 — Comptes & Outils

Avant d'écrire la moindre ligne de code, crée tous les comptes dont tu as besoin. Chaque service a un plan gratuit suffisant pour démarrer.

■ 1.1 — Crée ton compte Supabase

Va sur supabase.com → New Project → Nom : **social-seller** → Région : choisir la plus proche de tes utilisateurs (EU West ou US East). Note ta **Project URL** et ta clé **anon public** dans un fichier `.env.local`.

■ 1.2 — Crée ton compte Railway

Va sur railway.app → Login with GitHub. Tu déploieras ici ton serveur de webhooks au Sprint 2.

■ 1.3 — Installe les outils locaux

```
# Node.js (si pas installé)
# Télécharge sur nodejs.org – version LTS

# Expo CLI
npm install -g expo-cli

# Claude Code
npm install -g @anthropic-ai/claude-code

# Vérifie les installations
node --version
expo --version
```

■ Si tu es sur Mac, installe Homebrew d'abord (`brew.sh`), puis : `brew install node`

Jour 2 — Initialiser le Projet Expo

■ 2.1 — Crée le projet avec Claude Code

Ouvre ton terminal, navigue dans ton dossier de travail, puis lance Claude Code et utilise ce prompt :

```
Crée un projet Expo avec TypeScript et Expo Router appelé "social-seller".
```

```
Structure de dossiers souhaitée :
```

```
social-seller/  
app/  
  (auth)/  
    login.tsx  
    activate.tsx  
  (app)/  
    dashboard.tsx  
    inbox.tsx  
    orders.tsx  
    catalogue.tsx  
    _layout.tsx  
    index.tsx  
  components/  
    ui/  
      inbox/  
      orders/  
    lib/  
      supabase.ts  
      types.ts  
      constants.ts  
    hooks/  
    assets/
```

```
Installe ces dépendances :
```

```
- @supabase/supabase-js  
- expo-router  
- expo-secure-store  
- react-native-safe-area-context  
- @tanstack/react-query  
- zustand
```

```
Configure aussi le fichier .env.example avec :
```

```
EXPO_PUBLIC_SUPABASE_URL=  
EXPO_PUBLIC_SUPABASE_ANON_KEY=  
WEBHOOK_SERVER_URL=
```

■ 2.2 — Configure Supabase dans le projet

```
Dans lib/supabase.ts, crée le client Supabase avec :
```

```
- Lecture des variables d'environnement EXPO_PUBLIC_SUPABASE_URL et  
EXPO_PUBLIC_SUPABASE_ANON_KEY  
- Session persistée avec expo-secure-store
```

- Export du client typé

Crée aussi lib/types.ts avec les types TypeScript pour :

- Tenant, User, Agent, Conversation, Message, Product, Order
- Tous les enums de statut (OrderStatus, ConversationStatus, SocialNetwork)

■ Claude Code va générer tous les fichiers. Lis-les avant de les valider — prends l'habitude de comprendre ce qu'il génère, même brièvement.

Jours 3–4 — Schéma Base de Données Multi-Tenant

C'est l'étape la plus importante de tout le Sprint 1. Un bon schéma multi-tenant avec Row Level Security (RLS) évite des semaines de problèmes plus tard.

■ 3.1 — Génère le schéma SQL avec Claude Code

Génère le schéma SQL complet pour Supabase d'une app multi-tenant appelée Social Seller.

Tables à créer :

1. tenants

- id (uuid, primary key)
- name (text)
- whatsapp_number (text, unique)
- email (text, nullable)
- country (text)
- currency (text, default 'XOF')
- logo_url (text, nullable)
- plan (enum: 'starter', 'pro', 'enterprise')
- status (enum: 'active', 'suspended', 'cancelled')
- created_at, updated_at

2. users (extension de auth.users de Supabase)

- id (uuid, references auth.users)
- tenant_id (uuid, references tenants)
- role (enum: 'super_admin', 'merchant', 'agent')
- full_name (text)
- whatsapp_number (text)
- is_active (boolean, default true)
- created_at

3. social_connections

- id, tenant_id
- network (enum: 'whatsapp', 'facebook', 'tiktok')
- account_id (text)
- account_name (text)
- access_token (text, encrypted)
- status (enum: 'connected', 'disconnected')
- created_at, updated_at

4. conversations

- id, tenant_id
- network (enum: 'whatsapp', 'facebook', 'tiktok')
- external_conversation_id (text)
- customer_name (text)
- customer_external_id (text)
- assigned_agent_id (uuid, nullable, references users)
- status (enum: 'new', 'in_progress', 'resolved')
- last_message_at (timestamp)
- created_at

5. messages

- id, tenant_id, conversation_id
- direction (enum: 'inbound', 'outbound')
- content (text)
- media_url (text, nullable)
- sent_by_user_id (uuid, nullable)
- external_message_id (text)
- created_at

6. products

- id, tenant_id
- name (text)
- description (text, nullable)
- price (numeric)
- stock_quantity (integer)
- unit (text, default 'piece')
- image_url (text, nullable)
- is_active (boolean, default true)
- created_at, updated_at

7. orders

- id, tenant_id, conversation_id (nullable)
- customer_name (text)
- customer_whatsapp (text, nullable)
- status (enum: 'new', 'confirmed', 'preparing', 'shipped', 'delivered', 'cancelled')
- total_amount (numeric)
- created_by_user_id (uuid, references users)
- cancel_reason (text, nullable)
- created_at, updated_at

8. order_items

- id, order_id, product_id
- quantity (integer)
- unit_price (numeric)

Ajoute les politiques Row Level Security (RLS) pour chaque table :

- Les users ne peuvent voir que les données de leur tenant_id
- Le super_admin peut voir toutes les données

- Les agents ne voient que les conversations qui leur sont assignées ou non assignées

Génère aussi les index sur : `tenant_id`, `status`, `created_at` pour chaque table.

■ 3.2 — Applique le schéma dans Supabase

Dans ton dashboard Supabase → SQL Editor → colle et exécute le SQL généré. Vérifie dans Table Editor que toutes les tables sont créées correctement.

■■ Ne mets jamais de vraies clés d'API ou tokens dans ton code. Utilise toujours les variables d'environnement (`.env.local`). Ajoute `.env.local` à ton `.gitignore` avant le premier commit.

Jours 5–6 — Authentification & Onboarding WhatsApp

■ 5.1 — Écran de connexion

Crée l'écran de connexion `app/(auth)/login.tsx` pour Social Seller.

Fonctionnalités :

- Champ numéro de téléphone (format international avec sélecteur de pays)
- Champ mot de passe
- Bouton "Se connecter"
- Lien "Première connexion ? Activer mon compte"
- Authentification via Supabase Auth (email/password, mais l'email = `numéro@socialseller.app` en interne)
- Gestion des erreurs (mauvais mot de passe, compte suspendu)
- Redirection vers dashboard si déjà connecté
- Design : fond blanc, couleur principale `#2d6a4f`, boutons arrondis
- Compatible iOS et Android

■ 5.2 — Création de tenant par Super Admin

Crée l'écran et la logique pour qu'un Super Admin puisse créer un nouveau tenant.

Écran `app/(admin)/create-tenant.tsx` :

- Formulaire : nom boutique, numéro WhatsApp (format international), pays, plan
- Email optionnel
- Bouton "Créer et envoyer invitation WhatsApp"

Logique (`lib/tenants.ts`) :

- Insérer le tenant en base avec status `'active'`
- Créer le user merchant lié à ce tenant dans `auth.users` de Supabase
- Générer un token d'activation (uuid) stocké dans une table `activation_tokens` avec expiration à 48h
- Appeler la fonction `sendActivationWhatsApp(whatsapp_number, activation_token)`

La fonction `sendActivationWhatsApp` doit appeler l'API WhatsApp Business (endpoint : `process.env.WEBHOOK_SERVER_URL/send-activation`)

avec le template de message suivant :

'Bonjour ! Votre compte Social Seller est prêt.'

Activez-le ici : [LIEN_ACTIVATION]
Ce lien expire dans 48h.'

■ 5.3 — Activation du compte (US-04)

Crée l'écran d'activation `app/(auth)/activate.tsx`

Paramètre URL : token (ex: `/activate?token=xxxx-xxxx`)

Logique :

- Vérifier que le token existe dans `activation_tokens` et n'est pas expiré
- Si expiré : afficher message d'erreur + bouton "Renvoyer un lien"
- Si valide : afficher formulaire de création de mot de passe (mot de passe + confirmation, min 8 caractères)
- À la validation : mettre à jour le mot de passe dans Supabase Auth, marquer le token comme utilisé (`used_at = now()`), rediriger vers l'écran de setup boutique

Crée aussi l'endpoint dans le webhook server pour renvoyer un nouveau lien (POST `/resend-activation`) qui génère un nouveau token et renvoie le WhatsApp.

■ 5.4 — Setup profil boutique (US-05)

Crée l'écran de configuration du profil boutique `app/(app)/setup-boutique.tsx`

Champs :

- Nom de la boutique (pré-rempli depuis le tenant)
- Upload de logo (utilise `expo-image-picker` + Supabase Storage)
- Description courte (max 200 caractères)
- Secteur d'activité (liste déroulante : Mode, Alimentaire, Beauté, Electronique, Autre)
- Pays (pré-rempli)
- Devise (liste : XOF, XAF, MAD, GHS, NGN, USD, EUR)
- Bouton "Enregistrer"

À la sauvegarde :

- Upload le logo dans Supabase Storage bucket 'logos' (dossier `tenant_id/`)
- Mettre à jour la table `tenants` avec toutes les infos
- Rediriger vers le dashboard principal

■ Claude Code va générer beaucoup de code. Concentre-toi sur le faire tourner avant de le perfectionner. Un code qui marche > un code parfait qui ne tourne pas.

Jours 7–8 — Gestion Abonnement & Notifications WhatsApp

■ 7.1 — Suspension / Résiliation (US-02)

Crée les fonctions de gestion d'abonnement dans `lib/subscriptions.ts` :

```
1. suspendTenant(tenantId: string, reason: string)
- Met le status du tenant à 'suspended'
- Désactive tous les users du tenant (is_active = false)
- Envoie un message WhatsApp au commerçant :
"Votre compte Social Seller a été suspendu. Motif : [RAISON].
Contactez-nous pour régulariser votre situation."
```

```
2. cancelTenant(tenantId: string)
- Met le status à 'cancelled'
- Planifie la suppression des données à J+30 (created_at d'un job)
- Envoie un message WhatsApp au commerçant
```

```
3. reactivateTenant(tenantId: string)
- Remet le status à 'active'
- Réactive les users
- Envoie message WhatsApp de confirmation
```

Crée aussi l'écran admin `app/(admin)/tenants.tsx` listant tous les tenants avec leurs statuts et boutons d'action (suspendre / résilier / réactiver).

■ 7.2 — Rappels de renouvellement (US-03)

Crée un système de rappels d'abonnement dans le webhook server (Node.js).

Ajoute dans la table tenants :

- `subscription_expires_at` (timestamp)
- `next_renewal_reminder_sent` (timestamp, nullable)

Crée un endpoint GET `/cron/renewal-reminders` qui :

1. Récupère tous les tenants actifs dont `subscription_expires_at` est dans 7j, 3j ou 1j
2. Pour chaque tenant, envoie un message WhatsApp via l'API Meta :
"Bonjour [NOM_BOUTIQUE] ! Votre abonnement Social Seller expire dans [X] jours.
Renouvelez maintenant pour ne pas perdre l'accès : [LIEN_PAIEMENT]"
3. Met à jour `next_renewal_reminder_sent`

Configure ce cron job pour tourner chaque jour à 9h UTC via Railway Cron.

Jours 9–10 — Tests & Sprint Review

■ 9.1 — Tests de l'isolation multi-tenant

Génère des tests pour vérifier l'isolation multi-tenant dans Supabase.

Tests à écrire (Jest + Supabase client) :

1. Créer 2 tenants distincts (Tenant A et Tenant B)
2. Créer un user merchant pour chaque tenant
3. Vérifier que le merchant du Tenant A ne peut pas lire les données du Tenant B
4. Vérifier que le super_admin peut lire les données des deux tenants

5. Vérifier que la création d'une conversation lie bien le bon tenant_id

Utilise le client Supabase avec les credentials de chaque user pour simuler les requêtes.

■ 9.2 — Checklist Sprint Review

À la fin du sprint, vérifie chaque point avant de valider le sprint comme terminé :

- US-01 : Un Super Admin peut créer un tenant depuis l'interface
- US-01 : Le commerçant reçoit bien un message WhatsApp avec son lien d'activation
- US-02 : La suspension bloque l'accès au commerçant et à ses agents
- US-02 : La notification WhatsApp de suspension est envoyée
- US-03 : Le rappel J-7 est envoyé automatiquement via le cron
- US-04 : Le lien d'activation fonctionne et expire après 48h
- US-04 : Le renvoi de lien via WhatsApp fonctionne
- US-05 : Le profil boutique est sauvegardé avec logo uploadé
- Isolation tenant : Un merchant ne voit pas les données d'un autre tenant
- Les variables d'environnement sont bien configurées (pas de clés en dur)

■ Fais ta Sprint Review comme si tu étais ton propre client. Teste les scénarios d'erreur (token expiré, mauvais mot de passe, WhatsApp non reçu) pas seulement le happy path.

Ressources clés

Ressource	URL	Quand l'utiliser
Supabase Docs	supabase.com/docs	Setup BDD, Auth, RLS, Storage
WhatsApp Business API	developers.facebook.com/docs/whatsapp	Sprint 2 — connexion WhatsApp
Expo Docs	docs.expo.dev	Navigation, composants, build
Railway Docs	docs.railway.app	Déploiement serveur webhooks
Claude Code	docs.claude.ai/code	Génération de code assistée